

BMCS2063 Data Structures and Algorithms

Abstract Data Types (ADTs)

Chapter 2

Learning Outcomes

At the end of this lecture, you should be able to

- Explain the benefits of abstract data types (ADTs).
- Write ADT specifications.
- Implement ADTs using Java interfaces and classes.

Winning Strategies in Programming

Increase Productivity

- Projects can be finished on time
- More projects can be handled

Assure Quality

- Reliable: bug-free

How to ... *increase productivity*
and *assure quality*?

Code reuse

Code maintainability

To achieve *reuse*

Abstraction

- ADT specification

Encapsulation

- ADT implementation

To achieve *maintainability*

Encapsulation
/ Information
Hiding

- ADT implementation

Abstraction

- **Abstraction** is a fundamental concept in computer science and software engineering that involves
 - **simplifying complex** systems by hiding unnecessary details and exposing only the essential features.
 - to **manage complexity**, enhance understanding, and improve the efficiency of software development and usage.

Benefits

Manage Complexity:

- By focusing on **high-level concepts**: - helps manage the complexity of software systems, making them easier to design, develop, and maintain.

Enhance Reusability:

- promotes **code reuse** by allowing developers to create general-purpose components that can be used in different contexts.

Improve Maintainability:

- helps in **isolating changes**. When implementation details change, the high-level interface remains the same, reducing the impact on the rest of the system.

Facilitate Communication:

- High-level abstractions provide a common language for developers, designers, and stakeholders to **communicate effectively** about system functionality and design.

Benefits of

Abstraction

- Can focus on the abstract properties without worrying about how it is going to be implemented.

Encapsulation

- Can use the components without knowing the implementation details.

Abstraction & Encapsulation

Abstraction

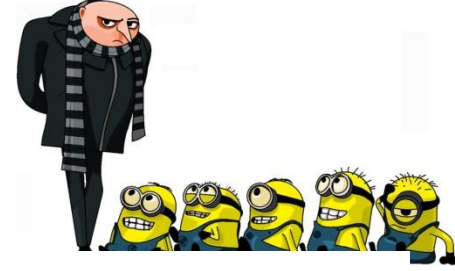
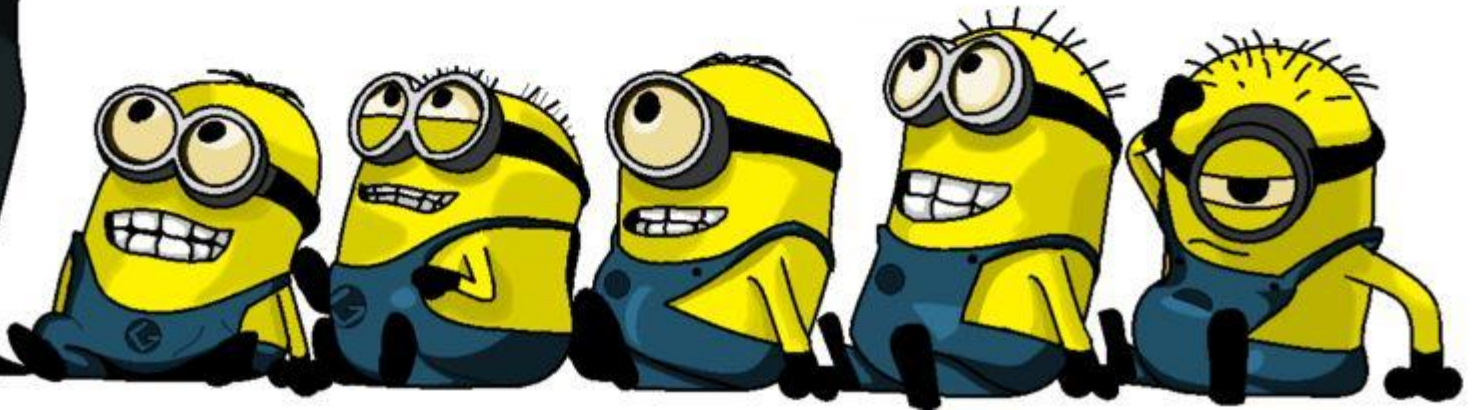
- Specifying the data type

Encapsulation

- Using the data type

2 sides of the same coin

Case Study



Gru's observation

- Code duplication – reinventing the wheel
- Not-so-maintainable code

MinionSoft's code duplication

- The list is a “thing” that appears in all of the software applications.
- It has specific characteristics
 - Data in the list is organized in a certain way
 - There are certain operations that are performed on the list
- The declaration for the array to hold the list elements and the coding for the operations were repeated in all the software applications.

MinionSoft's not-so-maintainable code

- If a bug is discovered, the changes to the code would need to be applied to every single software module or application which used a list.

How to...

- Enable reuse?
- Increase maintainability?

Solution Steps

1. Abstraction

- Identify the general/abstract properties and operations of the list.

□ An abstract data type

- Produce a specification of the list
 - What are the characteristics of the data?
 - What are the operations for manipulating the data?

Solution Steps (cont'd)

2. Encapsulation

- Implement the list in such a way that a programmer can use the list without knowing how it is implemented.

□ Information hiding

An *ADT*

An ADT specifies the **operations** that can be performed on the data and the **rules** for these operations, but it does not specify how these operations are implemented.

Key Characteristics of ADTs

1. **Abstraction**: ADTs focus on what operations are performed and the types of data involved, rather than how these operations are carried out.
2. **Encapsulation**: The details of the data representation and the implementation of operations are hidden from the user. Only the interface (the operations) is visible.
3. **Modularity**: ADTs promote modularity by allowing the implementation to change without affecting the code that uses the ADT.

ADT Specifications

- refers to the **definition** and **description** of an ADT, focusing on its **behavior** and **operations** rather than its implementation details.
- Written in a natural language (e.g. English) and are independent of any programming language.
- Used as specifications for concrete data types (i.e. the actual data types used in programs).

What to include in an ADT specification?

- ADT **title**
- Description of the **characteristics** (logical properties) of the data type
- Description of each operation:
 - Operation header: return type (if any), **operation name**, parameters (if any)
 - Brief **description** of what the operation does
 - **Precondition** (if any)
 - **Postcondition**
 - What is **returned** by the operation (if any)

Preconditions and Postconditions

Precondition

- A statement specifying the condition(s) that must be true before the operation is invoked.

Postcondition

- A statement specifying what is true after the operation is completed.

Problem: A Counter ADT

- Counter devices are used for counting things such as cars entering a parking lot, people taking numbers at the post-office, etc. A counter object would have a non-negative integer value representing the current count. It can be incremented, decremented, reset to zero and have its value read at any time.

Exercise 2.1

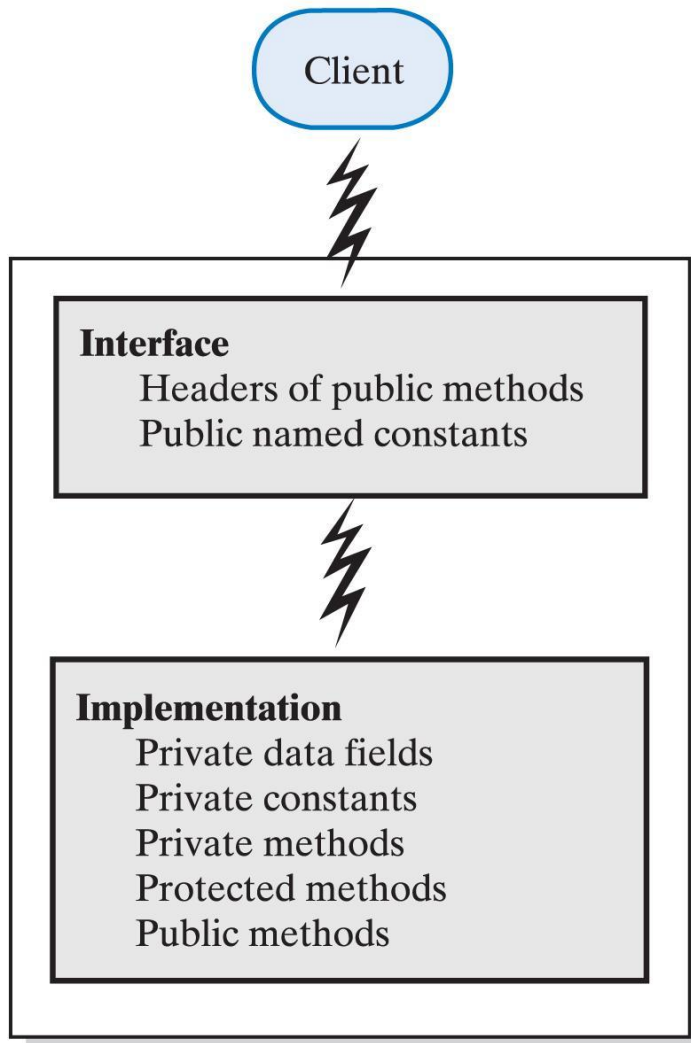


- Write the ADT specification for a counter whose instances would represent counter objects.

Implementing Encapsulation

- In object-oriented programming, encapsulation is achieved in a class by
 - ❑ Making its data fields private, and
 - ❑ Providing public methods for controlled access to and manipulation of the data fields

Data Abstraction



An interface provides well-regulated communication between a hidden implementation and a client.

Carrano (2011)

To implement an ADT in Java

1. Translate the ADT specification into a Java interface
2. Write a class which implements the Java interface

Sample Code: Counter ADT

In Chapter2\carpark\

- **CounterInterface.java**
- **Counter.java**
- **CarParkSystem.java**
 - A GUI application which simulates the use of the counter in a car park with 2 wings

Summary: To create an ADT

Step 1 Write the ADT specification

- Write an ADT specification which describes the characteristics of that data type and the set of operations for manipulating the data. **Should not include any implementation or usage details.**

Step 2 Implement the ADT

- a. Write a Java interface
 - Include all the operations from the ADT specification
- b. Write a Java class
 - This class implements the Java interface from a.
 - Determine how to represent the data
 - Implement all the operations from the interface

Step 3 Use the ADT in a client program or application

An ADT specification

- Defines the structure, behavior and operations of a “new” data type without specifying how those structure, behavior and operations are actually implemented

ADT: Rationale

- Separation of Concerns

- By separating the definition of the data structure from the implementation, we can use the new data structure in programs without regard for its implementation.
- Hence, the implementation can be changed (improved, updated, etc) and the client programs (i.e. the programs that use the data structure) will only have to change the name of the class that implements the new data structure.

Learning Outcomes

You should now be able to

- Explain the benefits of abstract data types (ADTs).
- Write ADT specifications.
- Implement ADTs using Java interfaces and classes.